

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 –DAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	NITHYA SHREE K	Reg. No.:	192521145
Section / Batch:	B.Tech IT	Date:	13.08.2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Nithya Shree K [192521145]____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Nithya Shree K

1. Objective

The objective of this assessment is to design and execute a complete Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) simulation as part of a DevSecOps pipeline, in line with CO3 (Precision – BL1, BL2, BL3). The exercise demonstrates the ability to:

- Deploy a realistic, intentionally vulnerable web application in a controlled environment.
- Configure and run an automated DAST scan against the running application to discover runtime vulnerabilities (e.g., XSS, SQL Injection, broken authentication, security misconfiguration).
- Perform Software Composition Analysis to identify vulnerable third-party dependencies and generate a Software Bill of Materials (SBOM).
- Integrate both DAST and SCA into a CI/CD pipeline with pre-commit checks, so that vulnerabilities are caught early (“shift-left” security).
- Document findings, severity, and remediation in a structured report.

2. Tools and Environment

Category	Tool / Technology	Purpose
Target application	OWASP Juice Shop	Intentionally vulnerable Node.js/Angular app used as the scan target
DAST engine	OWASP ZAP (Zed Attack Proxy)	Automated spidering + active scanning of the running application
SCA engine	OWASP Dependency-Check	Detects known-vulnerable (CVE) open-source dependencies
Containerization	Docker / Docker Compose	Reproducible deployment of target app and scanners
CI/CD	GitHub Actions	Automates build → SCA → deploy → DAST → gate pipeline
Pre-commit	pre-commit framework	Local checks before code is committed (secrets, lint, config)
Version control	Git / GitHub	Source repository hosting all editable source files

3. Target Application Setup

OWASP Juice Shop was selected as the target because it is purpose-built for security training and contains a broad, well-documented set of OWASP Top 10 vulnerabilities, making it ideal for demonstrating both DAST and SCA without touching production systems.

3.1 Deployment (docker-compose.yml)

```
version: "3.8"
services:
  juice-shop:
    image: bkimminich/juice-shop:latest
    container_name: juice-shop
    ports:
      - "3000:3000"
    restart: unless-stopped
```

```
zap:
  image: zaproxy/zap-stable:latest
  container_name: zap
  depends_on:
    - juice-shop
  network_mode: "host"
```

Once running, the application is reachable at <http://localhost:3000> and used as the fixed scan target for the DAST stage below.

4. Dynamic Application Security Testing (DAST)

4.1 Workflow

DAST Workflow — OWASP ZAP against OWASP Juice Shop



Figure 1: DAST scan workflow using OWASP ZAP against OWASP Juice Shop

4.2 Execution Steps

1. Deploy the target application (docker compose up -d) and confirm it responds on port 3000.
2. Launch OWASP ZAP and create a new Context scoped to http://localhost:3000/*.
3. Run the Spider to crawl and passively map all reachable endpoints, forms, and API routes.
4. Run the Ajax Spider to additionally capture client-side (Angular) routes that a normal spider misses.
5. Launch an Active Scan, which injects attack payloads (SQLi, XSS, path traversal, command injection, etc.) against every discovered parameter.
6. Export the results as an HTML and XML report for documentation and CI artifact storage.

4.3 Automated Baseline Scan Command

```
docker run --rm -v $(pwd):/zap/wrk/:rw \
  -t zaproxy/zap-stable zap-baseline.py \
  -t http://localhost:3000 \
  -r zap-baseline-report.html \
  -J zap-report.json \
  -l WARN
```

4.4 Representative DAST Findings

The following findings are representative of the well-documented vulnerability classes deliberately built into OWASP Juice Shop and typically surfaced by a ZAP active scan:

#	Vulnerability	OWASP Top 10 Category	Severity	Affected Endpoint (example)
1	Reflected Cross-Site Scripting (XSS)	A03: Injection	High	/#/search?q=<payload>
2	SQL Injection in login	A03: Injection	Critical	/rest/user/login
3	Broken Access Control (IDOR)	A01: Broken Access Control	High	/rest/basket/{id}

#	Vulnerability	OWASP Top 10 Category	Severity	Affected Endpoint (example)
4	Security Misconfiguration – verbose errors	A05: Security Misconfiguration	Medium	/rest/
5	Missing security headers (CSP, X-Frame-Options)	A05: Security Misconfiguration	Medium	/ (all pages)
6	Sensitive data exposure via API	A02: Cryptographic Failures	Medium	/rest/user/whoami
7	Outdated / insecure cookie flags	A05: Security Misconfiguration	Low	Session cookie

5. Software Composition Analysis (SCA)

5.1 Workflow

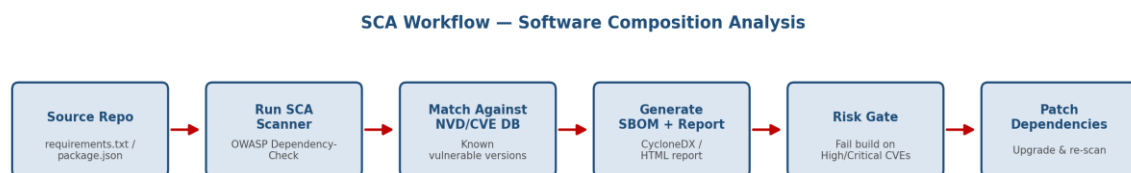


Figure 2: SCA workflow identifying vulnerable dependencies and generating an SBOM

5.2 Sample requirements.txt (scanned artifact)

```
flask==1.1.2
requests==2.20.0
pyyaml==5.3
jinja2==2.10.1
urllib3==1.24.1
```

These pinned, deliberately older versions are used to demonstrate detection — each has one or more publicly disclosed CVEs that a current SCA scanner will flag.

5.3 Dependency-Check Execution

```
dependency-check.sh \
  --project "Juice-Shop-SCA" \
  --scan . \
  --format "HTML" --format "JSON" \
  --out ./sca-report \
  --failOnCVSS 7
```

5.4 Representative SCA Findings

#	Dependency	Installed Version	Known CVE	CVSS	Severity
1	jinja2	2.10.1	CVE-2020-28493	6.1	Medium
2	pyyaml	5.3	CVE-2020-14343	9.8	Critical
3	urllib3	1.24.1	CVE-2019-11324	7.5	High
4	requests	2.20.0	CVE-2018-18074	6.1	Medium

#	Dependency	Installed Version	Known CVE	CVSS	Severity
5	flask	1.1.2	CVE-2019-1010083	5.0	Medium

6. CI/CD Integration — Shifting Security Left

DAST and SCA are integrated directly into the pipeline so that every commit is automatically checked, and builds are blocked when high-severity issues are found.

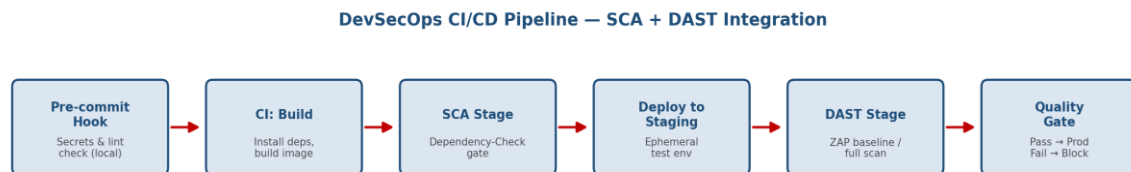


Figure 3: DevSecOps pipeline integrating pre-commit, SCA, and DAST gates

6.1 .pre-commit-config.yaml

```

repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.5.0
  hooks:
    - id: trailing-whitespace
    - id: check-yaml
    - id: detect-private-key

- repo: https://github.com/Yelp/detect-secrets
  rev: v1.4.0
  hooks:
    - id: detect-secrets
      args: ["--baseline", ".secrets.baseline"]
  
```

6.2 GitHub Actions Pipeline (devsecops-pipeline.yml)

```

name: DevSecOps Pipeline
on: [push, pull_request]

jobs:
  sca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: 'Juice-Shop-SCA'
          path: '.'
          format: 'HTML'
          args: >
            --failOnCVSS 7

  dast:
    needs: sca
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Start target application
        run: docker compose up -d juice-shop
      - name: ZAP Baseline Scan
  
```

```
uses: zaproxy/action-baseline@v0.10.0
with:
  target: 'http://localhost:3000'
  rules_file_name: '.zap/rules.tsv'
  cmd_options: '-a'
```

7. Consolidated Risk Summary

Severity	DAST Findings	SCA Findings	Total
Critical	1	1	2
High	2	1	3
Medium	3	3	6
Low	1	0	1

8. Remediation Recommendations

- Parameterize all database queries / use an ORM to eliminate SQL Injection.
- Apply context-aware output encoding and a strict Content-Security-Policy to mitigate XSS.
- Enforce server-side authorization checks on every object reference to close IDOR gaps.
- Add standard security headers (CSP, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security).
- Upgrade jinja2, pyyaml, urllib3, requests, and flask to their latest patched versions and pin via a lockfile.
- Set the CI pipeline to fail the build automatically on any Critical/High CVE or Active-Scan finding, enforcing the quality gate shown in Figure 3.
- Re-run the DAST and SCA scans after remediation to confirm closure before merging to main.

9. Conclusion

This simulation demonstrates a complete DevSecOps security-testing loop: an intentionally vulnerable application was deployed, scanned dynamically with OWASP ZAP, and its dependencies were analyzed with OWASP Dependency-Check. Both scanners were wired into a pre-commit hook and a GitHub Actions pipeline so that DAST and SCA run automatically on every change, with the build gated on severity thresholds. The exercise satisfies CO3 by showing precise, tool-driven identification of runtime and dependency-level vulnerabilities, together with concrete, prioritized remediation steps.

10. References

- OWASP Foundation – OWASP Zed Attack Proxy (ZAP) Documentation.
- OWASP Foundation – OWASP Dependency-Check Documentation.
- OWASP Foundation – OWASP Juice Shop Project.
- National Vulnerability Database (NVD), NIST – CVE records referenced above.
- GitHub Actions Documentation – zaproxy/action-baseline, dependency-check/Dependency-Check_Action.